Student name: MOLAPO SETSUMI
Student number: 202322042

Introduction

This project management plan sets out the foundation for the Moderated Learning Management System (LMS) required by the CS4433 Software Engineering assignment. The plan covers the initial feasibility study, the work breakdown structure, the schedule, and the risk register, which together form the Week 1 deliverable. Its purpose is to demonstrate that the project is achievable within the seven-week timeframe and to establish a disciplined approach to managing scope, time, and quality.

The LMS is intended to support four roles—administrator, lecturer, Head of Department, and student—while enforcing moderation of marks before release. Unlike a simple database, the system must record every change to marks and provide secure access for each role. These requirements shape the technical and operational considerations of the plan and influence the way risks are managed.

The project follows the Waterfall model with V-Model verification and validation. Each phase produces a documented deliverable, and progress is reviewed at three gates with the supervisor. This plan therefore acts not only as a roadmap but also as a baseline against which later phases will be checked for consistency and traceability.

By setting out feasibility, effort estimates, and risks at the start, the plan ensures that the project begins with realistic expectations. It also makes clear how scope will be controlled and how progress will be monitored, so that the final system can be delivered on time and to specification.

1.1 Assumptions

The project runs from Monday 5 October to Sunday 22 November 2026, giving seven weeks in total. Gate reviews with the supervisor are scheduled at the end of Weeks 2, 3, and 6, and the viva will take place in Week 7. These checkpoints

provide opportunities to confirm that requirements, design, and implementation remain aligned.

Workload is estimated at about 15 hours per week, for a total of 105 hours. All roles—project manager, analyst, architect, developer, tester, and technical writer—are carried out individually. Effort estimates include time to draft the report sections as each phase ends, so documentation is produced continuously rather than left to the final week.

The system will run locally on a school computer without requiring internet access. A menu-driven console interface is acceptable to the school and is chosen for simplicity and reliability. Online payment, video conferencing, automated grading, plagiarism detection, and web or mobile interfaces are explicitly excluded from scope, in line with the brief.

Finally, the plan assumes that the school will provide basic technical support for installation and backup, but not ongoing development. Version 1.0 is designed for a single computer, and any extension to a local network would be considered a future enhancement beyond the current assignment.

2. Feasibility Study

2.1 Technical Feasibility

The project relies on mature, free tools: C++17, CMake, SQLite, Git, GoogleTest/Catch2, and cppcheck. SQLite is well suited to the school's environment: it stores all data in a single file, requires no server, and can handle the tens of thousands of rows generated by 800 students' enrolments, submissions, and marks. Performance will be tested against a measurable target: a user can log in and open their assignment list within 2 seconds

with 800 students stored. Passwords will be stored as salted hashes using a recognised library (e.g. libsodium).

Risks: handling SQLite resources correctly, and unreliable file locking if the database is shared over a network.

Mitigation: wrap database handles in RAII classes, and restrict version 1.0 to a single computer, as permitted by the brief.

Conclusion: technically feasible with clear risk management.

2.2 Operational Feasibility

The system mirrors existing workflows: lecturers mark, HoDs moderate, and students see marks only after release. The lecturer would prefer students to see marks immediately, and I resolve this conflict in the requirements phase. A simple menu-driven interface with clear prompts and plain-language error messages will ensure usability for non-technical staff. Each role will be supported by a short manual.

Conclusion: operationally feasible if the interface remains simple and documentation is clear.

2.3 Economic Feasibility

All development tools are free, hardware is already available, and there are no licensing or hosting costs. The system replaces paper or spreadsheet marking, enforces late penalties consistently, and provides a permanent audit trail of mark changes.

Conclusion: economically feasible with no cash outlay.

2.4 Legal Feasibility

The system stores personal data of minors, so compliance with Lesotho's Data Protection Act, 2011 (Act No. 5 of 2012) is

mandatory, even though the Commission created by the Act is reportedly not yet fully operational. Measures include role-based access, secure password storage, and audit logging. The licences of the chosen tools (SQLite public domain, libsodium ISC, GoogleTest BSD-3, CMake BSD-3, and cppcheck GPL, used for development only) permit school use without restriction.

Conclusion: legally feasible if privacy and security are treated as non-negotiable requirements.

2.5 Schedule Feasibility

The seven-week plan is demanding but achievable because the scope is limited and documentation carries significant weight. The estimated effort is 105 hours, but Weeks 2 to 4 run above the planned 15 hours a week (see section 4.5), so some weekend work will be needed. Risks include schedule overrun and gate failures.

Mitigation: prioritise Must requirements, draft report sections as phases end, set up CI early, and keep a one-day buffer before each gate.

Conclusion: feasible with disciplined scope control.

2.6 Recommendation

I recommend that the project goes ahead. All five areas are feasible, so the main question is whether I can deliver on time. That depends on four conditions: keep the interface simple, treat security as a Must requirement, hold the scope to the brief, and keep the report up to date as I go. If I fall more than a day behind on a critical task, I will drop Could and then Should requirements before touching any Must requirement.

3. Work Breakdown Structure and Effort

The project is split into nine phases and 39 work packages. Effort is estimated in hours and totals 105. Each estimate includes the

time to draft the matching part of the final report, so the report is written as I go. The estimates are my own judgement based on the size of each deliverable in the brief, and I will compare them with my actual hours at each gate. Packages 1.5 to 1.7 are the gate reviews: before each one I update the risk register and check that the documents agree with each other. Package 9.2 is the traceability matrix. I start it at Gate 1 and complete and check it at the end.

ID	Work package	Effort (hours)
	1. Project management	10
1.1	Feasibility study	2
1.2	Work breakdown structure and schedule	2
1.3	Initial risk register	2
1.4	Finalise and submit project plan	1
1.5	Gate 1 review and risk register update	1
1.6	Gate 2 review and risk register update	1
1.7	Gate 3 review and risk register update	1
	2. Requirements engineering	15
2.1	Stakeholder analysis	3
2.2	Conflict and ambiguity resolution	3
2.3	Software Requirements Specification	4
2.4	Use case diagram	2
2.5	Three fully dressed use case specs	3
	3. System design	19
3.1	Architecture and component diagram	3
3.2	Class diagram and design patterns	5
3.3	State, activity and sequence diagrams	5
3.4	Database design (ERD, dictionary, SQL)	5
3.5	Consolidate design document	1
	4. Implementation	25
4.1	Repository, CMake and CI setup	3
4.2	Data-access layer and database	5

ID	Work package	Effort (hours)
4.3	User hierarchy, authentication, security	4
4.4	Business logic (penalty, moderation)	7
4.5	Audit log	2
4.6	Console user interface	3
4.7	Code review and cleanup	1
	5. Verification and validation	13
5.1	Test plan (V-Model mapping)	3
5.2	Unit and integration tests, coverage	5
5.3	System and acceptance tests, defect log	5
	6. Configuration management and QA	4
6.1	Branching strategy and tagging	2
6.2	Static analysis, metrics, self review	2
	7. Deployment	5
7.1	Deployment diagram	1
7.2	Install guide, manuals, release notes	3
7.3	Package and tag release v1.0	1
	8. Maintenance	6
8.1	CR1 classification and impact analysis	2
8.2	Implement CR1 (release v1.1)	2
8.3	Regression testing	2
	9. Completion	8
9.1	Professional, legal, ethical issues	2
9.2	Complete and verify traceability matrix	2
9.3	Final report assembly	2
9.4	Viva preparation	2
	Total	105

4. Schedule

The schedule starts on Monday 5 October 2026, so Week 7 ends on Sunday 22 November. Durations are in calendar days because I will work some weekends. Figure 1 is the task-level Gantt chart. Red bars are on the critical path, blue bars have float, diamonds are milestones and the thin lines are dependencies.

4.1 Gantt chart



Figure 1: Task-level Gantt chart

.2 Task schedule

ID	Task	Days	Start	Finish	Predecessors	Float	Critical
1.1	Feasibility study	2	Mon 5 Oct	Tue 6 Oct	–	0 d	Yes
1.2	Work breakdown structure and schedule	2	Wed 7 Oct	Thu 8 Oct	1.1	2 d	No
1.3	Initial risk register	2	Wed 7 Oct	Thu 8 Oct	1.1	2 d	No
1.4	Finalise and submit project plan	1	Fri 9 Oct	Fri 9 Oct	1.2, 1.3	2 d	No
P0	◆ Plan submitted (end of Week 1)	–	Sun 11 Oct	Sun 11 Oct	1.4	–	–
2.1	Stakeholder analysis	3	Wed 7 Oct	Fri 9 Oct	1.1	0 d	Yes
2.2	Conflict and ambiguity resolution	2	Sat 10 Oct	Sun 11 Oct	2.1	0 d	Yes
2.3	Software Requirements Specification	3	Mon 12 Oct	Wed 14 Oct	2.2	0 d	Yes
2.4	Use case diagram	2	Mon 12 Oct	Tue 13 Oct	2.2	2 d	No
2.5	Three fully dressed use case specs	2	Thu 15 Oct	Fri 16 Oct	2.3, 2.4	1 d	No
5.1	Test plan (V-Model mapping)	2	Sat 17 Oct	Sun 18 Oct	2.5	21 d	No

ID	Task	Days	Start	Finish	Predecessors	Float	Critical
3.1	Architecture and component diagram	3	Thu 15 Oct	Sat 17 Oct	2.3	0 d	Yes
1.5	Gate 1 review and risk register update	1	Sun 18 Oct	Sun 18 Oct	2.5, 3.1	0 d	Yes
3.2	Class diagram and design patterns	2	Mon 19 Oct	Tue 20 Oct	3.1, 1.5	0 d	Yes
3.3	State, activity and sequence diagrams	2	Wed 21 Oct	Thu 22 Oct	3.2	1 d	No
3.4	Database design (ERD, dictionary, SQL)	3	Wed 21 Oct	Fri 23 Oct	3.2	0 d	Yes
3.5	Consolidate design document	1	Sat 24 Oct	Sat 24 Oct	3.3, 3.4	0 d	Yes
1.6	Gate 2 review and risk register update	1	Sun 25 Oct	Sun 25 Oct	3.5	0 d	Yes
4.1	Repository, CMake and CI setup	2	Mon 26 Oct	Tue 27 Oct	1.6	0 d	Yes
6.1	Branching strategy and tagging	1	Mon 26 Oct	Mon 26 Oct	1.6	12 d	No
4.2	Data-access layer and database	3	Wed 28 Oct	Fri 30 Oct	4.1	0 d	Yes

ID	Task	Days	Start	Finish	Predecessors	Float	Critical
4.3	User hierarchy, authentication, security	2	Sat 31 Oct	Sun 1 Nov	4.2	2 d	No
4.4	Business logic (penalty, moderation)	4	Sat 31 Oct	Tue 3 Nov	4.2	0 d	Yes
4.5	Audit log	1	Wed 4 Nov	Wed 4 Nov	4.4	3 d	No
4.6	Console user interface	4	Wed 4 Nov	Sat 7 Nov	4.3, 4.4	0 d	Yes
4.7	Code review and cleanup	1	Sun 8 Nov	Sun 8 Nov	4.5, 4.6, 6.1	0 d	Yes
5.2	Unit and integration tests, coverage	4	Wed 4 Nov	Sat 7 Nov	4.4	1 d	No
5.3	System and acceptance tests, defect log	5	Mon 9 Nov	Fri 13 Nov	4.7, 5.2, 5.1	0 d	Yes
6.2	Static analysis, metrics, self review	3	Mon 9 Nov	Wed 11 Nov	4.7	2 d	No
7.1	Deployment diagram	2	Mon 9 Nov	Tue 10 Nov	3.5	3 d	No
7.2	Install guide, manuals, release notes	3	Wed 11 Nov	Fri 13 Nov	4.7	0 d	Yes
7.3	Package and tag release v1.0	1	Sat 14 Nov	Sat 14 Nov	5.3, 6.2, 7.1, 7.2	0 d	Yes

ID	Task	Days	Start	Finish	Predecessors	Float	Critical
R1	◆ Release v1.0	–	Sat 14 Nov	Sat 14 Nov	7.3	–	–
1.7	Gate 3 review and risk register update	1	Sun 15 Nov	Sun 15 Nov	7.3	0 d	Yes
8.1	CR1 classification and impact analysis	2	Mon 16 Nov	Tue 17 Nov	1.7	0 d	Yes
8.2	Implement CR1 (release v1.1)	1	Wed 18 Nov	Wed 18 Nov	8.1	0 d	Yes
8.3	Regression testing	1	Thu 19 Nov	Thu 19 Nov	8.2	0 d	Yes
R2	◆ Release v1.1	–	Thu 19 Nov	Thu 19 Nov	8.3	–	–
9.1	Professional, legal, ethical issues	3	Mon 16 Nov	Wed 18 Nov	1.7	1 d	No
9.2	Complete and verify traceability matrix	3	Mon 16 Nov	Wed 18 Nov	5.3	1 d	No
9.3	Final report assembly	2	Fri 20 Nov	Sat 21 Nov	R2, 9.1, 9.2	0 d	Yes
9.4	Viva preparation	1	Sun 22 Nov	Sun 22 Nov	9.3	0 d	Yes

4.3 Dependencies and milestones

Every dependency in the table is finish-to-start: a task starts after its predecessors finish. Four points shape the plan.

- Implementation cannot start until the Gate 2 review (1.6) is done. This is a course rule.
- Architecture (3.1) starts once the SRS is drafted, because the brief lets design begin in Week 2. It is only confirmed at Gate 1, and class design (3.2) waits for that gate.
- The test plan (5.1) is derived from the use case specifications (2.5). In the V-Model, system and acceptance tests are specified from the requirements, not from the code.
- The installation guide and manuals (7.2) are written while system testing (5.3) runs, so I will need a quick update pass if a defect fix changes behaviour.

Milestones: plan submitted on Sunday 11 October; Gate 1 review on Sunday 18 October; Gate 2 review on Sunday 25 October; release v1.0 on Saturday 14 November; Gate 3 review on Sunday 15 November; release v1.1 on Thursday 19 November. The brief puts the viva in Week 7, and I will agree the exact date with my lecturer.

4.4 Critical path and float

The critical path is the longest chain of dependent tasks, so a delay to any task on it delays the finish. In this plan it runs:

1.1 Feasibility study → 2.1 Stakeholder analysis → 2.2 Conflict resolution → 2.3 SRS → 3.1 Architecture → Gate 1 review (1.5) → 3.2 Class diagram → 3.4 Database design → 3.5 Design document → Gate 2 review (1.6) → 4.1 Repository and CI → 4.2 Data-access layer → 4.4 Business logic → 4.6 Console interface → 4.7 Code review → 5.3 System and acceptance tests → 7.3 Release v1.0 → Gate 3 review (1.7) → 8.1 Impact analysis → 8.2 Implement CR1 → 8.3 Regression tests → 9.3 Final report → 9.4 Viva preparation.

The installation guide and manuals (7.2) have no float either, because they must be finished before the release alongside the system tests. With Waterfall, most of the project is on the critical path, which is why scope control matters so much. A few tasks do have slack: the test plan (5.1) and branching set-up (6.1) have the most, followed by the deployment diagram (7.1, 3 days), the audit log (4.5, 3 days), and the use case diagram, authentication, static analysis and the rest of this plan (2 days each). The use case specifications, state and sequence diagrams, unit tests, professional issues and the traceability matrix each have 1 day. The float column in section 4.2 gives the exact figure for every

task. There is also a one-day gap before each gate review, which is where I will absorb small delays.

4.5 Planned effort per week

Week	Hours	Against 15 h	Main work
Week 1	13	2 under	Planning, stakeholder analysis, conflicts
Week 2	16	+1 over	SRS, use cases, test plan, architecture
Week 3	17	+2 over	Class, behavioural and database design
Week 4	17.5	+2.5 over	Repository, data layer, business logic
Week 5	14.5	0.5 under	Console interface, tests, code review
Week 6	13	2 under	System tests, QA, release v1.0
Week 7	14	1 under	Change request, report, viva
Total	105		

Weeks 2 to 4 (requirements finishing, design and the start of implementation) run above my 15-hour average, while Weeks 1 and 5 to 7 run below it. Because I cannot borrow time from later weeks, I will cover the peaks with weekend work, and if I fall more than a day behind I will cut Should and Could features first (risk R2 in section 5).

5. Risk Register

This is the initial version (Gate 0). Likelihood (L) and impact (I) are each scored from 1 to 5, and exposure is L × I. I rate exposure of 1 to 6 as Low, 7 to 12 as Medium and 13 to 25 as High. The owner column gives the role I am wearing when I handle each risk; since the project is individual, every role is me. I will review and re-score the register at Gates 1, 2 and 3 (tasks 1.5 to 1.7), adding new risks and closing old ones, and the updated version goes into the report appendix.

ID	Risk	Type	L	I	Exposure	Mitigation	Owner
R1	Stakeholder conflicts are left unresolved or resolved badly, giving an inconsistent SRS	Project	4	4	16 High	List every conflict and ambiguity in a table, record each resolution with its reason, and confirm the decisions with my supervisor at Gate 1	Analyst
R2	Schedule overrun because the documents take longer than planned	Project	4	4	16 High	Time-box each task using the estimates in section 3, drop Could then Should items first, keep a one-day gap before each gate, and write each report section as its phase ends	Project manager
R3	Scope creep: features outside the brief (payments, web interface, plagiarism checks) or extras beyond the Must requirements	Project	3	4	12 Medium	Treat the out-of-scope list in the brief as fixed, build Must items first, and only look at Should or Could items once the Must items pass their tests	Project manager

ID	Risk	Type	L	I	Exposure	Mitigation	Owner
R4	Gaps in C++ or SQLite C API knowledge cause delays or defects such as resource leaks and unhandled errors	People	3	4	12 Medium	Wrap SQLite handles and statements in RAII classes, build a thin log-in-and-query slice at the start of implementation, and read the documentation before each new kind of code	Developer
R5	Security flaw such as SQL injection or weak password storage	Technical	3	5	15 High	Parameterised queries only, salted hashes from a recognised library, no passwords in logs, static analysis, and tests that feed in malicious input	Developer
R6	Illness or personal circumstances reduce the time available	People	3	4	12 Medium	Do the heavy tasks early, use the lighter Weeks 1 and 5 to 7 as slack, and tell my supervisor early if I fall behind	Project manager

ID	Risk	Type	L	I	Exposure	Mitigation	Owner
R7	Late-penalty, extension or moderation rules are implemented wrongly, especially around deadlines	Technical	3	4	12 Medium	Define the rules exactly in the SRS (including what counts as a day), put them in a separate Strategy class, and test the values either side of the deadline and each penalty step	Developer
R8	Design, code, database and tests drift apart	Project	3	4	12 Medium	Keep the traceability matrix up to date from Gate 1 and compare the code with the class diagram and the tables with the ERD before each gate	Architect
R9	Gate 2 fails because the design is incomplete or the ERD is not in 3NF, which blocks implementation	Project	3	5	15 High	Check the class diagram against SOLID and the brief's list, normalise the ERD before the review, and ask for informal feedback during Week 3	Architect
R10	The system is too slow with 800 students	Technical	2	3	6 Low	Index the columns used in lookups and test with generated data against the 2-second target	Developer

ID	Risk	Type	L	I	Exposure	Mitigation	Owner
R11	Build or CI problems, such as a build that works on my machine but fails in CI or on another computer	Technical	3	3	9 Medium	Set up CMake and CI in the first days of implementation, pin library versions and document the build steps	Developer
R12	Line coverage of the business-logic layer ends up below 70%	Technical	3	3	9 Medium	Write unit tests alongside each business-logic class instead of at the end, and turn on coverage reporting (gcov/lcov) as soon as the first tests exist	Tester
R13	Loss of work or computer failure before submission or at the viva	Technical	2	5	10 Medium	Push to the hosted Git repository often, keep a second copy of the documents and a built copy of the program on a USB drive, and make sure the program runs offline	Project manager

ID	Risk	Type	L	I	Exposure	Mitigation	Owner
R14	The SQLite file is damaged or locked if it is used over a network share	Technical	3	4	12 Medium	Design v1.0 for one computer holding the database, state the limit in the installation guide, and make the data-access layer handle a locked database (busy timeout and a clear message)	Architect

The highest exposures are R1 and R2 (16), then R5 and R9 (15). Most of my early effort therefore goes into getting the requirements right, protecting the schedule, and building security and the design carefully the first time.

6. Monitoring and Control

- Weekly check. Each Sunday I compare the tasks finished and hours spent with this plan and write down any slippage and what I will do about it.
- Gate reviews. I meet my supervisor at the end of Weeks 2, 3 and 6. Before each review I update the risk register and check that the requirements, design, code and tests still match each other.
- Change control. After Gate 1, any change to the SRS is logged with its reason and a note of which designs, code and tests it affects.
- Version control. All documents and code go into Git. I will tag each gate (gate-1, gate-2, gate-3) and the releases (v1.0, v1.1).
- Slippage rule. If I am more than a day behind on a critical task, I cut Could and then Should requirements before touching any Must requirement.

7. References

1. Boehm, B.W. (1991) 'Software risk management: principles and practices', IEEE Software, 8(1), pp. 32–41.

2. Google (n.d.) GoogleTest user guide. Available at: https://google.github.io/googletest/ (Accessed: 4 October 2026).
3. ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018 Systems and software engineering: Life cycle processes: Requirements engineering. Geneva: ISO.
4. Kitware (n.d.) CMake documentation. Available at: https://cmake.org/documentation/ (Accessed: 4 October 2026).
5. Kingdom of Lesotho (2012) Data Protection Act, 2011 (Act No. 5 of 2012). Maseru: Government Gazette. Available at: https://lesotholii.org/akn/ls/act/2012/5 (Accessed: 4 October 2026).
6. libsodium (n.d.) libsodium documentation. Available at: https://doc.libsodium.org/ (Accessed: 4 October 2026).
7. Project Management Institute (2021) A guide to the project management body of knowledge (PMBOK guide). 7th edn. Newtown Square, PA: PMI.
8. Sommerville, I. (2016) Software engineering. 10th edn. Harlow: Pearson.
9. SQLite (n.d.) Appropriate uses for SQLite. Available at: https://www.sqlite.org/whentouse.html (Accessed: 4 October 2026).